

Open the black box — from any model

Deck 1 read the model directly (coefficients, tree splits). But a random forest, a boosting model, a neural net — you *can't* read those.

Model-agnostic tools work on *any* fitted model and answer two questions:

- ▶ **How much** does a feature matter? → **Permutation Feature Importance (PFI)**.
- ▶ **How** does a feature act on the prediction? → **feature effects (PDP / ICE)**.

Running example continues: a **bike-sharing** random forest (deck 1's data).

Outline

Permutation feature importance

Feature effects: PDP & ICE

Feature interactions

How much does a feature matter?

Make a feature useless and watch the error rise.

Permutation feature importance: the idea

Make feature X_j useless by permuting its column (shuffle its values across rows): same marginal distribution, but its link to the target is broken. Then see how much the error rises:

$$\text{PFI}_j = \underbrace{\mathbb{E}[L(\hat{f}(X_j \text{ permuted}), Y)]}_{\text{error after}} - \underbrace{\mathbb{E}[L(\hat{f}(X), Y)]}_{\text{baseline error}}.$$

- ▶ Big rise $\Rightarrow$ the model leaned on X_j . Near zero $\Rightarrow$ it didn't.
- ▶ **Permute, don't drop** — the model is already trained; we can't remove a feature without refitting a *different* model.
- ▶ Repeat the shuffle several times and average (it's random).

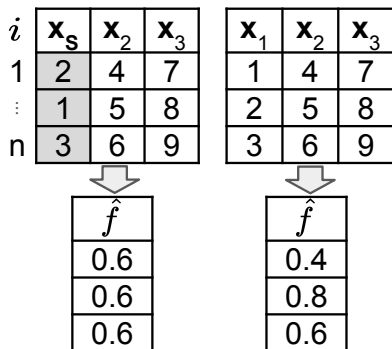
Permutation importance, step by step

i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$
1	2	4	7	1	4	7
$\vdots$	1	5	8	2	5	8
n	3	6	9	3	6	9

Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

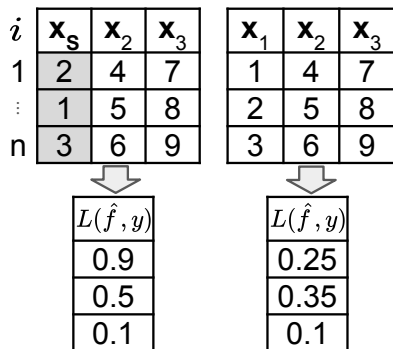
Permutation importance, step by step



Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step



Perturb (permute the feature) → **predict** on both versions → **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step

i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL
1	2	4	7	1	4	7	0.65
$\vdots$	1	5	8	2	5	8	0.15
n	3	6	9	3	6	9	0

$L(\hat{f}, y)$		$L(\hat{f}, y)$
0.9	-	0.25
0.5		0.35
0.1		0.1

Perturb (permute the feature) → **predict** on both versions → **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step

i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL
1	2	4	7	1	4	7	0.65
$\vdots$	1	5	8	2	5	8	0.15
n	3	6	9	3	6	9	0

= 0.267

Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step

1	i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL	
	1	2	4	7	1	4	7	0.65	
	$\vdots$	1	5	8	2	5	8	0.15	$= 0.267$
	n	3	6	9	3	6	9	0	
m	i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL	
	1	3	4	7	1	4	7	0.85	$\widehat{PFI}_s = \frac{1}{2} (0.267 + 0.4)$
	$\vdots$	2	5	8	2	5	8	0	
	n	1	6	9	3	6	9	0.35	$= 0.4$

Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

PERMUTATION FEATURE IMPORTANCE

$$\mathcal{R}_{\text{emp}}(\hat{f}, \tilde{\mathcal{D}}_{(k)}^S) - \mathcal{R}_{\text{emp}}(\hat{f}, \mathcal{D})$$

1	i	$\mathbf{x}_S$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL	
	1	2	4	7	1	4	7	0.65	
	$\vdots$	1	5	8	2	5	8	0.15	$= 0.267$
	n	3	6	9	3	6	9	0	
m	i	$\mathbf{x}_S$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL	
	1	3	4	7	1	4	7	0.85	
	$\vdots$	2	5	8	2	5	8	0	$= 0.4$
	n	1	6	9	3	6	9	0.35	

$\widehat{PFI}_S = \frac{1}{2} (0.267 + 0.4)$

3. Aggregation:

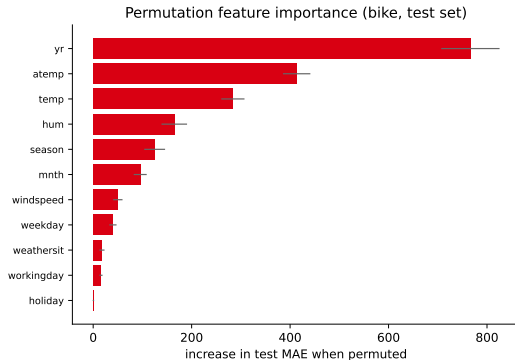
- Compute the loss for each observation in both data sets
- Take the difference of both losses ΔL for each observation
- Average this change in loss across all observations
- Repeat perturbation and average over multiple repetitions

PFI on the bike model

Permuting each feature on the **test set**,
measured as the rise in MAE:

- ▶ `yr`, `atemp`/`temp` dominate. By hand:
baseline MAE ≈ 455 ; shuffle `yr` $\rightarrow$ MAE ≈ 1225 ; PFI(`yr`) $\approx$ **770**.
- ▶ Calendar/weather minutiae barely matter.
- ▶ Error bars = variation over repeated shuffles.

Model-agnostic: same recipe would work on
boosting, an SVM, a neural net.



Train or test? They answer different questions

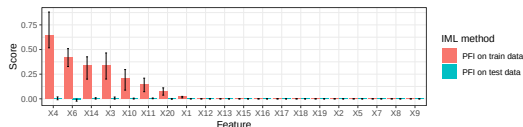
Predict-first: an xgboost model overfits noise. Which features does PFI flag — on train vs. on test?

Train or test? They answer different questions

Predict-first: an xgboost model overfits noise. Which features does PFI flag — on train vs. on test?

- ▶ **Train PFI** highlights whatever the model *overfit* to (spurious links).
- ▶ **Test PFI** shows what actually *generalizes* — here, nothing.

For “which features help the model generalize?”, compute PFI on **test** data.



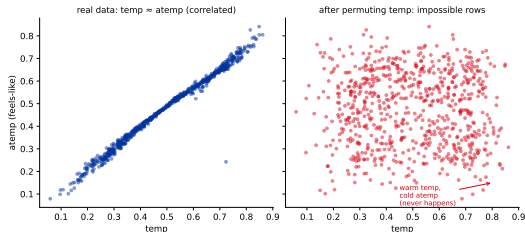
Illustrative example (not the bike model). Source: LMU IML (CC BY 4.0).

The correlation trap

Permuting a feature that is **correlated** with others creates **impossible rows** (e.g. a hot temp with a freezing atemp).

- ▶ The model is then scored on inputs it *never sees* in reality (extrapolation).
- ▶ $\Rightarrow$ PFI can **inflate** the importance of correlated features.

Note the flip: correlation *deflated* impurity importance (deck 1); here it *inflates* PFI. **Next three slides:** two repairs — permuting more carefully, and not permuting at all — and what they reveal about temp/atemp.



Bike: temp/atemp corr. 0.99; permuting temp makes impossible rows.

Repair 1 — Conditional Feature Importance (CFI)

The trap came from permuting **freely**. So don't: permute X_j only **among rows that look alike on the other features** — that is the “**conditional**” in the name.

- ▶ Group the rows by $\mathbf{x}_{-j}$ (e.g. the leaves of a small tree predicting X_j from the rest), then shuffle X_j *inside each group*.
- ▶ Warm days get swapped with warm days $\Rightarrow$ no freezing-atemp / hot-temp monsters.

It answers a different question. PFI: “does the model *use* this feature?” **CFI:** “does this feature carry information *beyond what the others already give me?*”

Consequence to expect: for two near-duplicate features, CFI sends **both** toward zero — each is redundant *given* the other. That is correct for the question CFI asks, and badly misleading if you read it as “temperature doesn't matter.”

On bike this drops temp from 284 to **18** and atemp from 413 to **39** MAE — while yr, which nothing can stand in for, barely moves (766 $\rightarrow$ 671).

Repair 2 — Leave-One-Covariate-Out (LOCO)

Deck's earlier warning was “we can't drop a feature without refitting a *different* model.”

Leave-One-Covariate-Out (LOCO; Lei et al., 2018) says: fine, **refit**.

$$\text{LOCO}_j = \underbrace{L(\hat{f}_{-j})}_{\text{model retrained without } j} - \underbrace{L(\hat{f})}_{\text{full model}}$$

- ▶ No permuting at all $\Rightarrow$ **no impossible rows, ever**.
- ▶ Cost: p **refits** instead of p shuffles.
- ▶ Question asked: “would I lose anything by **never collecting** this feature?”

The bike headline: PFI scores atemp at 413 — **second only to** yr — yet $\text{LOCO}(\text{atemp}) = -0.7$ MAE: drop it and the model gets *very slightly better*.

Nothing is broken — temp simply carries the same signal. **“Important” and “needed” are different words.**

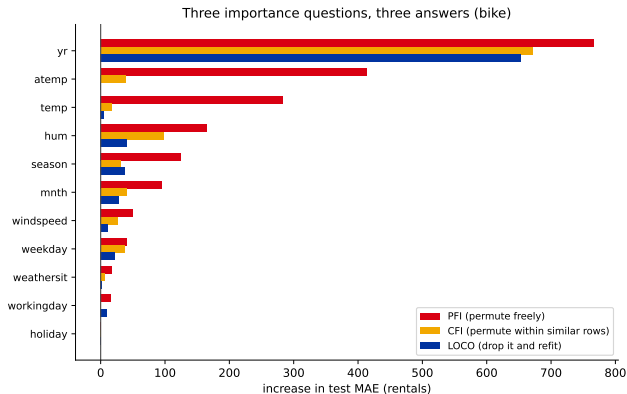
LOCO can go **negative**: removing a redundant feature can help by shortening the model's search.

Three questions, three answers

All three are measured in the **same unit** — rise in test MAE — so you can read them off one axis:

- yr: all three agree (766/671/653). **Nothing substitutes for it.**
- temp, atemp: huge under PFI, ≈ 0 under CFI and LOCO. **Each is only redundant because the other exists.**

A feature scoring high on PFI but ≈ 0 on LOCO is the signature of a **duplicated signal** — not of an unimportant feature.



And one more: SAGE, the fair split of performance

PFI, CFI and LOCO each remove **one** feature at a time — which is exactly why duplicated features confuse them. **SAGE** (*Shapley Additive Global importance*; Covert et al., 2020) removes features in **every possible subset** and gives each one its *average* contribution to the **loss**.

- ▶ That averaging-over-subsets is the **Shapley** idea — the whole of deck 3, but applied to the model's *error* instead of a single prediction.
- ▶ Because it is a fair split, the scores **add up** to the model's total loss reduction: importance as *shares of performance*.
- ▶ Redundant features **share** the credit rather than each looking essential (PFI) or each looking worthless (CFI/LOCO).

The catch: 2^P subsets. SAGE is **sam-pled**, not exact, and is by far the most expensive method in this deck. Library: `sage-importance`.

Pick by question, not by fashion:

PFI does the model use it?
CFI does it add new info?
LOCO do I need to collect it?
SAGE what is its fair share?

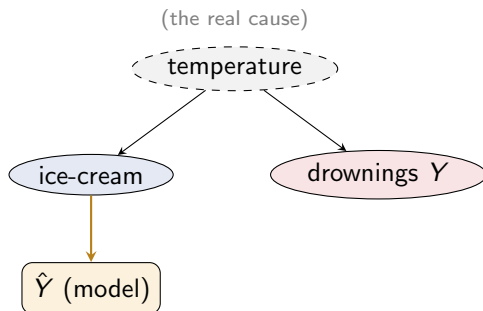
Important $\neq$ causal

Summer heat drives both **ice-cream sales** and **drownings**. A model predicting drownings from ice-cream sales finds ice cream **highly important**.

- ▶ PFI is right: the *model* relies on ice cream.
- ▶ But **banning ice cream won't stop drownings** — it isn't the cause.

Importance tells you what the **model uses**, not what *causes* the outcome.

In our data: *yr* is the #1 feature, yet the year doesn't *cause* rides — it proxies the service growing. (*Confounding — a different problem from the trap's impossible inputs.*)



Model uses the *proxy* (ice cream), not the cause (temperature).

From “how much” to “how”

Importance is one number; effects show the *shape* of a feature's influence.

Functional ANOVA: decomposing a prediction

Deck 1 introduced **main effects** and **interactions** in words. Here is the same split, written down: a baseline, each feature acting *alone*, then features acting *together*. It is the **functional ANOVA (fANOVA)** decomposition (Hooker, 2004):

$$\hat{f}(\mathbf{x}) = \underbrace{g_{\emptyset}}_{\text{baseline}} + \underbrace{\sum_j g_j(x_j)}_{\text{main effects}} + \underbrace{\sum_{j < k} g_{jk}(x_j, x_k)}_{\text{2-way interactions}} + \dots$$

Main effect g_j : how the prediction moves as x_j varies *alone* (averaging the rest).

Interaction g_{jk} : the leftover when x_j 's effect *depends on* x_k . Zero for an **additive** model (linear with no product terms, deck 1).

This one equation organises the whole deck: the effect plots estimate the g_j , and the interaction measures put a number on the g_{jk} .

Computing the components — and where the PDP fits

Work **order by order**: average out everything not in S , then subtract what the lower orders already explained.

$$g_S(\mathbf{x}_S) = \underbrace{\hat{f}_{S,\text{PD}}(\mathbf{x}_S)}_{\text{average out all features not in } S} - \sum_{V \subsetneq S} g_V(\mathbf{x}_V)$$

Unrolling the recursion:

$$g_{\emptyset} = \mathbb{E}[\hat{f}(X)]$$

$$g_j(x_j) = \hat{f}_{j,\text{PD}}(x_j) - g_{\emptyset}$$

$$g_{jk}(x_j, x_k) = \hat{f}_{jk,\text{PD}} - g_j - g_k - g_{\emptyset}$$

Read the second line again: the **centred PDP** is the fANOVA main effect g_j — not an approximation of it. That is why the PDP is *the* main-effect plot.

Two catches: the full decomposition needs 2^p PD-functions (hopeless past a handful of features), and the clean split **assumes independent features** — our temp/atemp problem again.

Why “ANOVA”? Because it splits the variance

Classical **ANOVA** splits the variance of an *outcome* into main effects plus interactions.
fANOVA does the same for the variance of a fitted *function*’s predictions.

If the features are **independent**, the components are uncorrelated, so the variance splits with **no covariance terms**:

$$\text{Var}[\hat{f}(X)] = \sum_S \text{Var}[g_S(X_S)]$$

Divide through by $\text{Var}[\hat{f}(X)]$ and the pieces are **fractions that sum to 1**.

So “importance” can also mean **share of variance** — yet another definition, on top of deck 1’s coefficients/impurity and this deck’s PFI. **Always say which one you mean.**

Sobol index — the share of prediction variance explained by one component:

$$S_V = \frac{\text{Var}[g_V(X_V)]}{\text{Var}[\hat{f}(X)]}$$

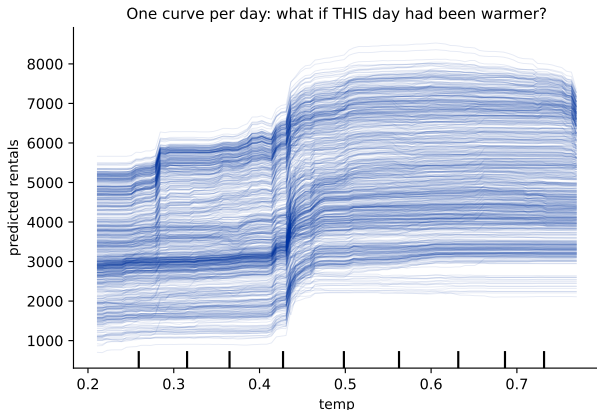
A main-effect S_j answers “how much of the model’s variability is feature j alone?”; the higher-order ones are what is left for pairs, triples, ...

Start local: one curve per observation (ICE)

Take **one day**. Hold everything about it fixed, sweep temp across its range, and record what the model predicts. That is one **individual conditional expectation (ICE)** curve — a what-if for that single day (Goldstein et al., 2015).

Do it for every day $\Rightarrow$ one curve each.

- The **level** differs: some days are busy, some quiet.
- The **shape** is what we care about — here almost every curve steps up around $\text{temp} \approx 0.45$.

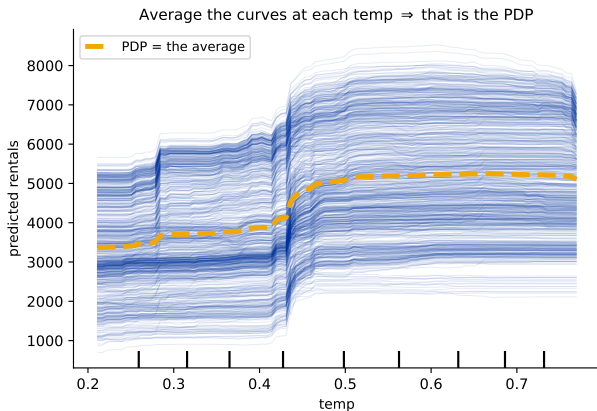


Average the ICE curves — and that *is* the PDP

At each value of temp, take the **mean over all the curves**. The result is the **partial dependence plot**.

$$\hat{f}_{j,\text{PD}}(x_j) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_j, \mathbf{x}_{-j}^{(i)})$$

- ▶ Centre it and you have g_j — the **fANOVA main effect** from two slides ago.
- ▶ One curve replaces 731: readable, but everything the individual curves disagreed about is now **gone**.



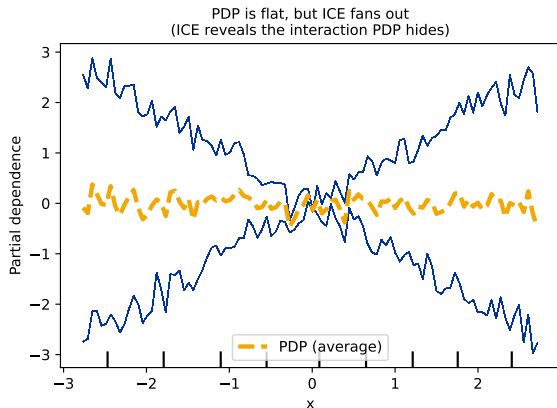
What the average can hide

Predict-first: this PDP (orange) is **flat**. Does the feature not matter?

What the average can hide

Predict-first: this PDP (orange) is **flat**. Does the feature not matter?

No. The ICE curves **fan out**: x 's effect **flips sign depending on another feature** — rising for half the rows, falling for the other half, so the opposing effects **cancel in the average**. The PDP hid that interaction; ICE exposes it — exactly the kind of interaction a *linear* model couldn't represent (deck 1).



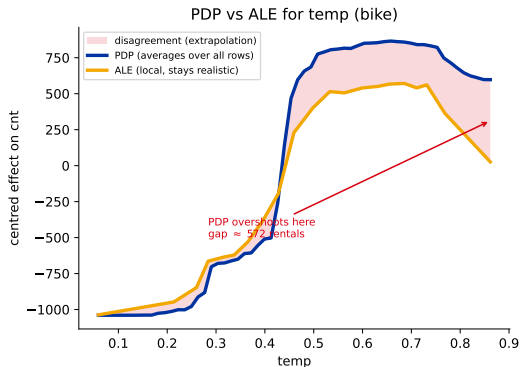
The visual test: *parallel* ICE curves = no interaction; curves that **fan out** = interaction. **Always look at the ICE before trusting a PDP.**

PDP shares PFI's correlation trap — and ALE fixes it

Look again at the averaging formula: it pairs every value of temp with every other row's features. With correlated features that builds the same **impossible rows** as the PFI trap — a hot temp on a freezing-atemp day.

ALE (Accumulated Local Effects; Apley & Zhu, 2020) fixes it: split the feature into small windows, and inside each window ask only how the prediction changes for the rows *that are actually there*. Accumulate those local changes $\Rightarrow$ a correlation-robust effect curve that never leaves the data.

But there is an obvious middle option we skipped — and it is a trap. Next slide.



Bike temp: PDP overshoots at the extremes; ALE stays realistic.

Three ways to hold the other features “fixed”

PDP — average over *all* rows:

$$\mathbb{E}_{X_{-j}} [\hat{f}(x_j, X_{-j})]$$

Builds impossible rows. **Extrapolates.**

M-plot — the obvious fix: average only over rows that *really have* that x_j :

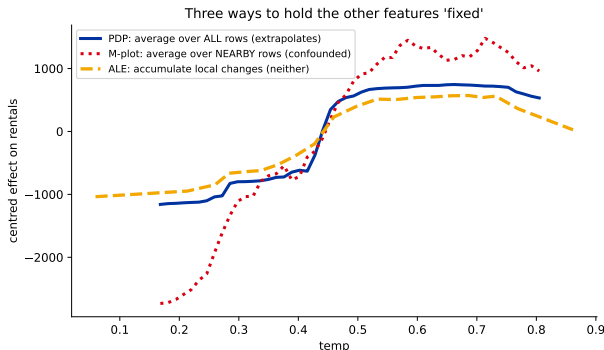
$$\mathbb{E}[\hat{f}(X) \mid X_j = x_j]$$

Never extrapolates — but warm days also have high atemp, season, mnth, so their effects get charged to temp.

Confounded.

ALE — inside each window take the *difference* in prediction; differencing cancels the neighbours' contribution.

Neither problem.



The M-plot spans ≈ 4200 rentals against ALE's ≈ 1600 : most of that steepness belongs to the *other* features, not to temp.

Correlated features $\Rightarrow$ use ALE.

When one feature's effect depends on another

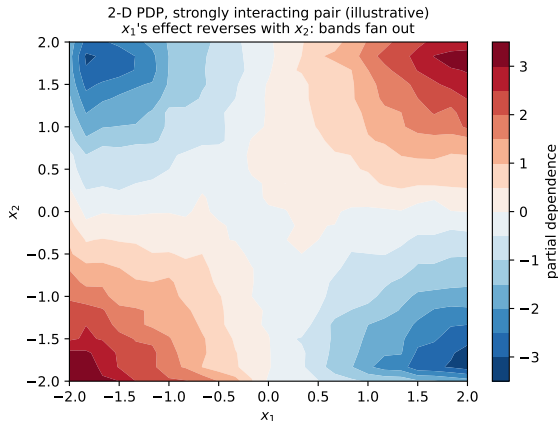
See it (the 2-D PDP), then put a *number* on it (the H-statistic).

Seeing an interaction: the 2-D PDP

Vary **two** features together and average over the rest — a partial-dependence **surface**.

- ▶ **No interaction** $\Rightarrow$ the bands are **parallel**: one feature's effect is the same at every value of the other.
- ▶ **Interaction** $\Rightarrow$ the bands **fan / bend**: here x_1 's effect **reverses** with the sign of x_2 (a clean $y = x_1x_2$ toy).

Illustrative on purpose — the bike model's *real* interactions are mild (we measure them next).



First: what exactly counts as “no interaction”?

To compare effects we first put them on the same footing — **centre** every PD-function by subtracting the baseline: $\hat{f}_{S,PD}^c = \hat{f}_{S,PD} - g_{\emptyset}$.

Definition. x_j and x_k **do not interact** if their joint effect is just the two separate effects added up:

$$\hat{f}_{jk,PD}^c(x_j, x_k) = \hat{f}_{j,PD}^c(x_j) + \hat{f}_{k,PD}^c(x_k)$$

So the **interaction is whatever is left over** when you subtract the two main effects from the joint one:

$$\underbrace{\hat{f}_{jk,PD}^c(x_j, x_k) - \hat{f}_{j,PD}^c(x_j) - \hat{f}_{k,PD}^c(x_k)}_{= g_{jk}(x_j, x_k)} \text{ — the fANOVA interaction component}$$

For an **additive** model this leftover is 0 at every point. All that remains is to turn “how big is the leftover?” into one number.

Friedman's H: the leftover as a fraction

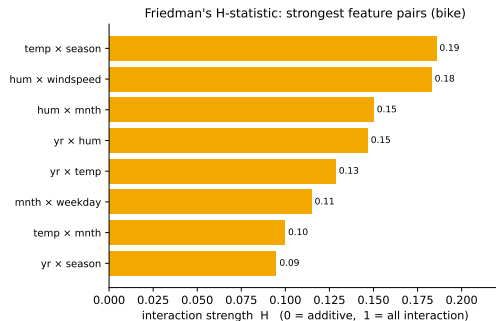
Take the **variance of the leftover**, relative to the variance of the joint effect (Friedman & Popescu, 2008 — *the same paper as RuleFit*, deck 1):

$$H_{jk}^2 = \frac{\text{Var} [\hat{f}_{jk}^c - \hat{f}_j^c - \hat{f}_k^c]}{\text{Var} [\hat{f}_{jk}^c]}$$

In practice, the sum over the rows:

$$H_{jk}^2 = \frac{\sum_i (\hat{f}_{jk}^c - \hat{f}_j^c - \hat{f}_k^c)^2}{\sum_i (\hat{f}_{jk}^c)^2} \quad (\text{at each row } i)$$

Dividing by the joint effect makes it a **share**, so $H \in [0, 1]$ and pairs are comparable: 0 = perfectly additive, $\rightarrow 1$ = almost all interaction.



All $H < 0.2$: the bike model is **mostly main effects**; **temp×season** ($H=0.19$) leads, but modestly.

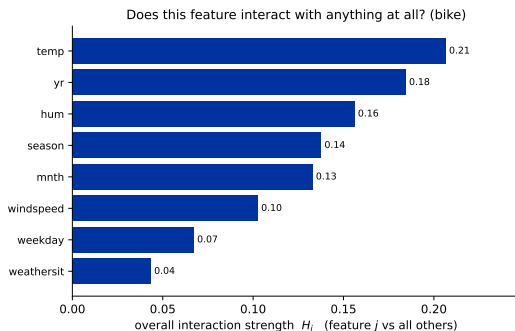
Does a feature interact with *anything*? (H_j) — and the caveats

Same construction, but split the world into **feature j** and **everything else** ($-j$) instead of two single features:

$$H_j^2 = \frac{\sum_i (\hat{f}^c(\mathbf{x}^{(i)}) - \hat{f}_j^c(x_j^{(i)}) - \hat{f}_{-j}^c(\mathbf{x}_{-j}^{(i)}))^2}{\sum_i (\hat{f}^c(\mathbf{x}^{(i)}))^2}$$

One number per feature: “*is this feature additive, or does it depend on the rest of the row?*” On bike, temp leads at 0.21 — the same feature whose curve bends (deck 1).

Caveats. (1) Costly: a k -way H needs $\approx 2^k$ PD-functions. (2) **Correlation inflates it** — we dropped atemp for that reason. (3) **No natural threshold** — read H as a *ranking*. (4) Not in sklearn: use pyartemis.



In sklearn

```
from sklearn.inspection import permutation_importance,
    PartialDependenceDisplay

# PFI on the TEST set -- works for ANY fitted model
r = permutation_importance(model, X_test, y_test, n_repeats=20,
    scoring="neg_mean_absolute_error")
r.importances_mean          # rise in MAE per feature = importance

# feature effects: PDP (average) + ICE (one line per row)
PartialDependenceDisplay.from_estimator(model, X, ["temp", "hum"])
    # PDP
PartialDependenceDisplay.from_estimator(model, X, ["temp"], kind="
    both")    # PDP + ICE
PartialDependenceDisplay.from_estimator(model, X, [("temp", "season"
    )])    # 2-D PDP (interaction)
```

Note what is **missing** here: sklearn has no ALE and no H-statistic. For those you leave sklearn — next slide.

Beyond sklearn: which library has what

You want...	Use...
PFI	sklearn (permutation_importance), daalex, alibi
CFI / LOCO	fippy, or a few lines by hand (group → shuffle; or drop → refit)
SAGE	sage-importance (not in sklearn)
PDP / ICE	sklearn (PartialDependenceDisplay), daalex, interpret (InterpretML)
ALE / M-plots	alibi, PyALE, effector, daalex (not in sklearn)
H-statistic	pyartemis (not in sklearn)
SHAP, counterfactuals	shap, dice-ml, alibi (deck 3)
Glass-box models	interpret (EBM), imodels (RuleFit, deck 1)

One-stop shops: daalex and alibi cover most of this deck in one API. **In R:** iml (Molnar's own) and DALEX.

Name trap: `pip install pyartemis`, but `import artemis` — plain artemis on PyPI is an unrelated package.

Recap

- ▶ **Model-agnostic** tools open any black box.
- ▶ **PFI** (*how much*): permute a feature, measure the error rise, on **test** data. Watch the **correlation trap**, and remember **important** $\neq$ **causal**.
- ▶ Four methods, **four different questions**: **PFI** (does the model use it?), **CFI** (new info beyond the others?), **LOCO** (must I collect it?), **SAGE** (fair share). Bike: $\text{atemp} = 413$ on PFI, -0.7 on LOCO.
- ▶ **fANOVA** (Hooker, 2004) organises it all: prediction = baseline + main effects + interactions, with g_j = the **centred PDP**; under independence it splits the **prediction variance** (the **Sobol index**).
- ▶ **ICE** (*how*): one what-if curve per row. **Average them and that is the PDP** — readable, but it can hide a sign-flipping interaction. Of the three ways to average, **PDP** extrapolates and **M-plots** confound, so with correlated features use **ALE** (Apley & Zhu, 2020).
- ▶ The **2-D PDP** shows an interaction; **Friedman's H** measures it as the **leftover after removing the main effects**: pairwise H_{jk} , or H_j for “does j interact with anything?” (bike: mostly main effects).

Next: SHAP — one method for a *single* prediction that also rolls up to global importance and effects.